

Career Orbit

Local-First Autonomous AI Job-Search Agent & Technical Architecture Guide

1. THE ELEVATOR PITCH (UNDER 30 SECONDS)

"I built Career Orbit, a local-first, autonomous AI job-search agent. It solves the most tedious part of job hunting by ingesting your resume, extracting your exact skills and experience, and autonomously scraping the web to find highly relevant, direct-apply roles. It ranks the jobs based on your specific profile—all without relying on expensive, privacy-invasive third-party LLMs."

2. THE PROBLEM VS. THE SOLUTION

The Problem

Modern job searching is broken. Candidates spend 80% of their time scrolling through generic job boards, dealing with recruiters, and filtering out promoted "ghost jobs." They have to manually match their skills to job descriptions over and over.

The Career Orbit Solution

We flip the model. The user provides their resume once, and the agent does the scrolling, filtering, and ranking autonomously.

- **Time-Saving:** Replaces hours of manual searching with a single click.
- **High Signal-to-Noise:** Strict skill matching filters out irrelevance.
- **Direct Apply:** Pulls direct ATS links (bypassing middle-men).

3. TECHNICAL ARCHITECTURE

COMPONENT	TECHNOLOGY & ARCHITECTURE DETAILS
Frontend	Built with React and Vite for an ultra-fast, modern, glassmorphic UI.
Backend	Powered by a lightweight Node.js and Express server.
Resume Engine	Uses pdf-parse and mammoth to extract text directly in memory. Custom NLP / RegEx engines parse roles, skills, location, and experience locally without third-party APIs.
Scraping Pipeline	Custom data-ingestion pipeline pulling jobs from public ATS endpoints (Greenhouse, Ashby, Workable, Lever) and a native LinkedIn guest scraper.
Scoring Algorithm	Weighted candidate-job matching model: Match Score = (Role × 40%) + (Skills × 30%) + (Location × 20%) + (Experience × 10%)
Deployment	Hosted on Render using a unified <code>npx vite build</code> pipeline that compiles the React SPA and serves static assets directly via Express.

4. LIVE PRODUCTION DATA FLOW

FLOW How does the app function end-to-end when a user hits the live URL?

- **Static Serving:** Render delivers the Express server, which serves pre-built React static bundle (`dist/` folder).
- **In-Memory Processing:** Uploaded resumes are handled via Express `multer` with Memory Storage. Files reside in RAM buffer during execution and are never written to physical disk, maintaining high performance and strict privacy.
- **State Management:** Uses a lightweight local JSON store (`state.json`) in memory for temporary persistence without heavy database overhead.
- **Concurrent Fetching:** Clicking "Search" triggers parallel asynchronous requests (`Promise.all()`) across ATS endpoints and LinkedIn guest scrapers, aggregating hundreds of listings in milliseconds.
- **Real-time Ranking:** Aggregated listings pass through the weighted scoring engine and return sorted JSON payloads to the frontend instantly.

5. DSA & ALGORITHMIC DEEP DIVE

Q1 How do you efficiently match skills against job descriptions? (String Matching)

Rather than a naive brute-force search ($O(N \times M)$), Career Orbit tokenizes the candidate's resume into known technical skills. When evaluating job descriptions, skills are stored in a **Set** for $O(1)$ lookups. Computing the skill-match score boils down to a fast **Set Intersection** between parsed candidate skills and job requirements.

Q2 How do you prevent showing duplicate job postings? (Deduplication / Hashing)

To prevent duplicates across concurrent scrapers, the backend computes a normalized key: `hash(normalizedCompany + normalizedTitle)` (stripping special characters and whitespace). These keys are tracked in a **Hash Map / Set** during aggregation. Existing keys are rejected in $O(1)$ time.

Q3 How do you sort and rank the jobs efficiently? (Sorting Algorithms)

Jobs are sorted by calculated "Match Score" using JavaScript's native `Array.prototype.sort()`, which executes **TimSort** (hybrid Merge/Insertion Sort) with $O(N \log N)$ time complexity and stable performance.

Q4 How does the backend process large PDFs without memory leaks? (Space Complexity)

Uploaded files are constrained to 8MB via `multer`. Using short-lived RAM Buffer objects trades controlled space for fast execution. Extracted buffers are quickly garbage-collected, preventing runtime memory leaks on Node.js.

6. GENERAL FAQ & INTERVIEW PREP

FAQ Why didn't you use ChatGPT/OpenAI for parsing?

To build a fully deterministic, fast, zero-cost parser. RegEx and keyword matching allow infinite scaling, zero API dependencies, instant response times, and 100% data privacy.

FAQ How do you bypass LinkedIn scraping blockers?

By querying LinkedIn's public guest API endpoints instead of scraping authenticated HTML DOM. This yields structured JSON data cleanly without requiring user sessions or risking IP/account bans.

FAQ What was the hardest deployment bug you encountered?

Configuring Vite build steps alongside the Express backend on Render. Render's build runner aggressively stripped devDependencies and cached lockfiles. Solved by restructuring package.json dependencies and using direct npx execution scripts.

7. KEY VALUE PROPOSITIONS

100% Free & Local-First: Zero external API costs or subscription paywalls. Runs entirely deterministic logic.

Privacy First: Resumes remain strictly on local memory / runtime buffers and are never sent to third-party LLMs for model training.

No Auto-Apply Spam: Gives users direct links to official ATS job portals, preserving candidate reputation while ensuring full control over applications.

8. FUTURE ROADMAP

1. **Automated Cover Letters:** Integrated lightweight local LLM (e.g., via Ollama/ONNX) to generate tailored cover letters locally.
2. **Email Digest Notifications:** Scheduled cron job delivering daily top-matched roles directly to candidate inboxes.
3. **Analytics & Conversion Dashboard:** Application funnel tracking to auto-tune matching weights based on interview callback rates.